

第 2 集 · 约 18 分钟

协议核心：所有权、Result 与 trait

加密函数的一行签名，藏着 Rust 的三条规矩

配套阅读：书的第 2 章



这一集你会学到

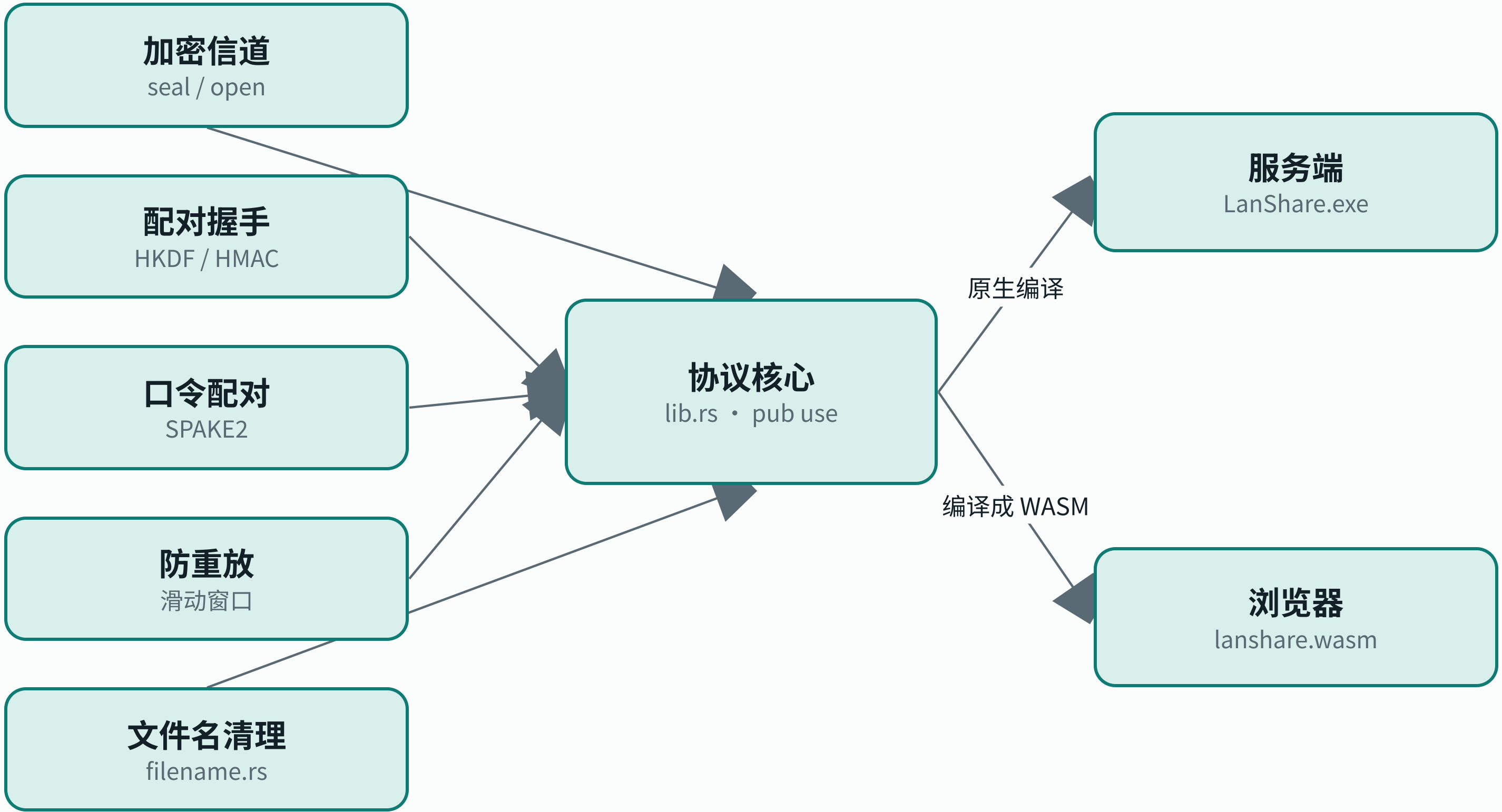
1. 所有权与借用：&[u8]、&[u8; 32]、Vec<u8> 各是什么意思
2. Result 和 ?：Rust 没有异常，会不会失败写在返回类型里
3. trait 是 Rust 的接口，它的方法要先 use 进来才能调
4. 怎么证明测试本身靠谱：独立算出的已知向量 + 变异测试

一个问题

加密密钥少了一个字节，程序什么时候才发现？

Rust 版：编译的时候。密钥参数的类型是 `&[u8; 32]`，长度写在类型里，31 个字节根本传不进去。

项目的心脏



四种“一串字节”

写法	含义	打个比方
<code>[u8; 32]</code>	正好 32 个字节的数组，长度是类型的一部分	一个固定 32 格的盒子
<code>&[u8; 32]</code>	借用一个 32 字节数组（只读，不拿走）	借来看看那个盒子
<code>&[u8]</code>	借用任意长度的一段字节（切片）	借来看一段字节，长度运行时才知道
<code>Vec<u8></code>	自己拥有的、可增长的字节数组	自己买的伸缩箱子

动手：值会搬家

```
1 let s1 = String::from("hi");
2 let s2 = s1;           // 搬家
3 println!("{}", s1);    // 还想用 s1?
4 let s3 = &s2;           // 借用：只看不拿
5 }                       // 作用域结束
```

变量（栈上）
(空)

数据（堆上）
已释放

离开作用域：主人 s2 被清理，它拥有的“hi”跟着释放。不需要垃圾回收。

对比 Python: `b = a` 只是给同一个对象多起一个名字，`a` 照样能用。

seal 和 open

```
crates/lanshare-proto/src/channel.rs
```

```
1 use chacha20poly1305::aead::{Aead, KeyInit, Payload};
2 use chacha20poly1305::{ChaCha20Poly1305, Key, Nonce};
3 // ...
4 /// 加密并附上认证标签。返回 `明文长度 + 16` 字节。
5 pub fn seal(key: &[u8; 32], ctr: u64, aad: &[u8], plain: &[u8]) -> Vec<u8> {
6     ChaCha20Poly1305::new(Key::from_slice(key))
7         .encrypt(&nonce(ctr), Payload { msg: plain, aad })
8         .expect("ChaCha20-Poly1305 只有在明文超过 256 GiB 时才会加密失败")
9 }
10
11 /// 验证并解密。任何一处不对（密钥、计数器、aad、密文）都返回 [`ProtoError::Decrypt`]。
12 pub fn open(key: &[u8; 32], ctr: u64, aad: &[u8], sealed: &[u8]) -> Result<Vec<u8>, ProtoError> {
13     ChaCha20Poly1305::new(Key::from_slice(key))
14         .decrypt(&nonce(ctr), Payload { msg: sealed, aad })
15         .map_err(|_| ProtoError::Decrypt)
16 }
```

异常 vs Result

Python (用 cryptography 库的写法)

```
1 try:
2     plain = aead.decrypt(nonce, sealed, aad)
3 except InvalidTag:
4     return error(403)
```

Rust v2 · server/channel.rs

```
1 let plain = proto::open(&keys.c2s, ctr, &proto::req_aad(pa
2     .map_err(|_| StatusCode::FORBIDDEN)?;
```

自己的错误类型

```
crates/lanshare-proto/src/lib.rs
```

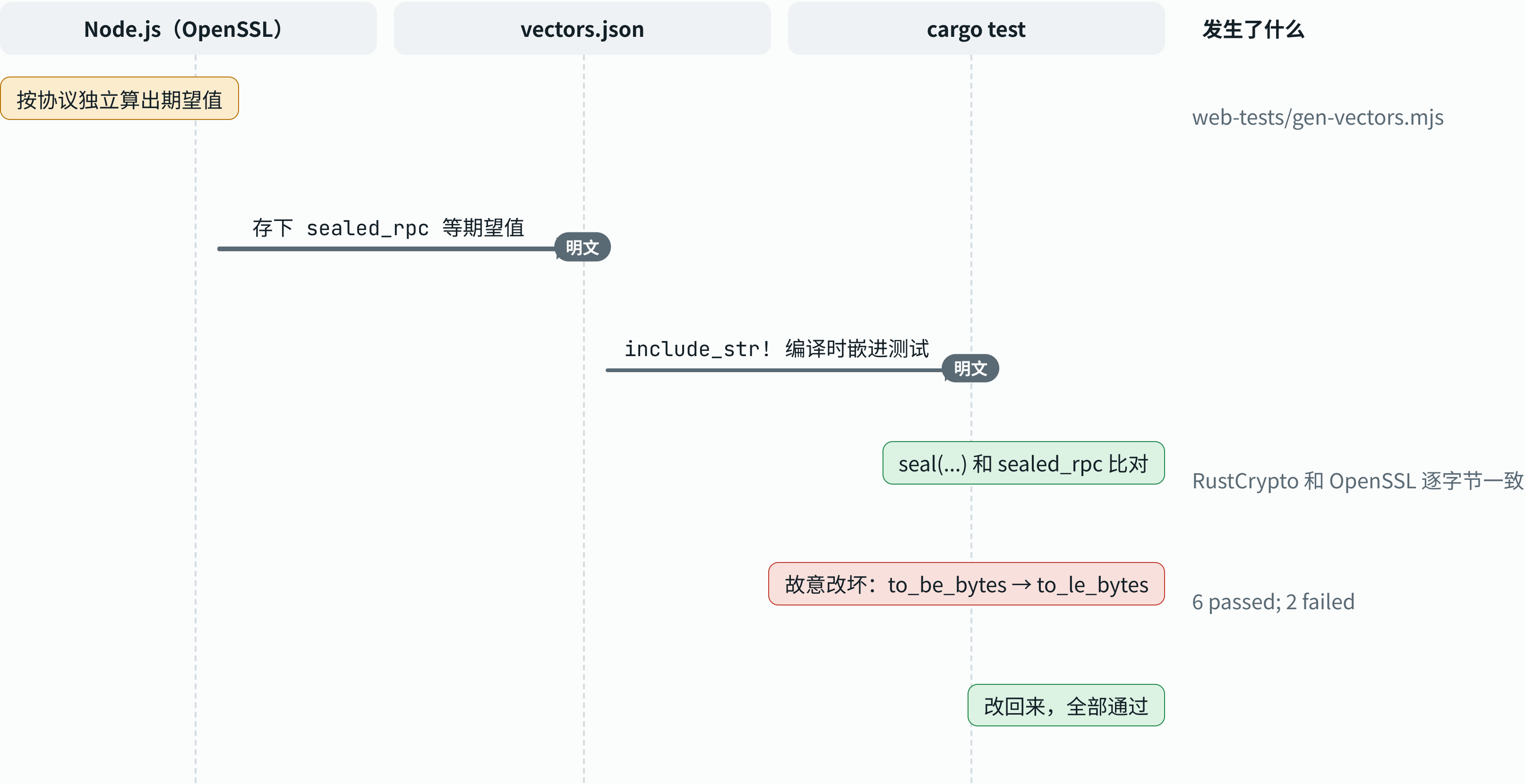
```
1  #[derive(Debug, Clone, Copy, PartialEq, Eq)]
2  pub enum ProtoError {
3      /// AEAD 认证失败：密钥、计数器、aad 或密文任意一处不对。
4      Decrypt,
5      /// PAKE 对端消息格式不对。
6      Pake,
7  }
8
9  impl std::fmt::Display for ProtoError {
10     fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
11         match self {
12             ProtoError::Decrypt => write!(f, "解密失败"),
13             ProtoError::Pake => write!(f, "口令配对消息无效"),
14         }
15     }
16 }
```

动手：改哪都解不开



演示用的是浏览器自带的 AES-GCM。LanShare 实际用 ChaCha20-Poly1305，两者都是“认证加密”：改一个字节、换一个附加数据都会解密失败，原理相同。nonce = 4 个零字节 + 8 字节计数器（和 LanShare 一样）。

期望值来自 “别人”



踩坑：特洛伊源码

● ● ● Windows PowerShell

```
PS D:\lanshare> cargo test -p lanshare-proto
```

```
error: unicode codepoint changing visible direction of text present in comment
```

选择题

`seal` 的密钥参数为什么写成 `&[u8; 32]`，而不是 `&[u8]`？

- A `&[u8; 32]` 比 `&[u8]` 运行得更快
- B 长度是类型的一部分，传错长度编译就不过
- C 两种写法没区别，只是写法习惯
- D 这样函数结束时，会把调用方的密钥释放掉

选择题

seal 的密钥参数为什么写成 `&[u8; 32]`，而不是 `&[u8]`？

A `&[u8; 32]` 比 `&[u8]` 运行得更快

这里的理由不是速度，而是让编译器保证密钥一定是 32 字节。

B 长度是类型的一部分，传错长度编译就不过

对。把规则写进类型里，31 个字节的密钥在编译阶段就被拦下，不用等到运行时。

C 两种写法没区别，只是写法习惯

有区别。`&[u8]` 的长度运行时才知道，`&[u8; 32]` 的长度编译时就定死了。

D 这样函数结束时，会把调用方的密钥释放掉

借用不拿走所有权。调用方把 `&key` 给出去之后，自己还能接着用。

选择题

这行代码末尾的 `?` 做了什么？

```
1 let key = state.0.finish(peer_msg).map_err(|_| ProtoError::Pake)?;
```

A 出错时程序直接崩溃退出

B 相当于 Python 里 except 之后写 pass，把错误吞掉继续往下走

C 表示 key 可能是空的，用之前还要再判断一次

D 出错就换成 ProtoError::Pake，立刻从当前函数返回；成功就把值取出来赋给 key

选择题

这行代码末尾的 `?` 做了什么？

```
1 let key = state.0.finish(peer_msg).map_err(|_| ProtoError::Pake)?;
```

A 出错时程序直接崩溃退出

`?` 不会让程序崩溃，它把错误交给调用方，由调用方决定怎么处理。

B 相当于 Python 里 `except` 之后写 `pass`，把错误吞掉继续往下走

正好相反。`?` 不吞错误，Err 一定会交给调用方。它更像 Python 里不写 `try`、让异常往上冒，只是 Rust 把这件事写进了签名。

C 表示 `key` 可能是空的，用之前还要再判断一次

过了 `?` 这一行，`key` 就是 Ok 里的那个值本身。出错的情况已经提前 `return` 了，走不到这里。

D 出错就换成 `ProtoError::Pake`，立刻从当前函数返回；成功就把值取出来赋给 `key`

对。`?` 就是“出错就提前 `return`”的简写，而函数的返回类型 `Result` 把这件事写在了明处。

选择题

漏写了 `use chacha20poly1305::aead::Aead;`，会怎样？

A 能编译，运行到 encrypt 那一行才报错

B 能编译，编译器会自动帮你 use 进来

C 编译失败，提示找不到 encrypt 方法，还会告诉你缺哪个 trait

D 连 ChaCha20Poly1305::new 也一起找不到了

选择题

漏写了 `use chacha20poly1305::aead::Aead;`，会怎样？

A 能编译，运行到 `encrypt` 那一行才报错

那是 Python 鸭子类型的思路：有没有这个方法，跑到才知道。Rust 在编译时就查清楚了。

B 能编译，编译器会自动帮你 `use` 进来

编译器只会在报错里提示你该 `use` 哪个 `trait`，不会替你改代码。

C 编译失败，提示找不到 `encrypt` 方法，还会告诉你缺哪个 `trait`

对。`encrypt`、`decrypt` 定义在 `Aead` 这个 `trait` 里，`trait` 不 `use` 进来，它的方法就调不了。

D 连 `ChaCha20Poly1305::new` 也一起找不到了

`new` 来自另一个 `trait KeyInit`，它的 `use` 还在，所以 `new` 没事，出问题的是 `encrypt` 和 `decrypt`。

想一想

Python v1 比对口令用 `secrets.compare_digest`，**Rust v2 校验握手标签用** `verify_slice`。两个版本都没用 `==`，为什么？

想一想

Python v1 比对口令用 `secrets.compare_digest`，Rust v2 校验握手标签用 `verify_slice`。两个版本都没用 `==`，为什么？

普通的 `==` 比到第一个不同的字节就返回，耗时会泄露“前几个字节对了”。攻击者反复测量响应时间，可以一个字节一个字节地猜出正确的标签。`compare_digest` 和 `verify_slice` 都是常量时间比较：不管哪里不同，耗时都一样。在这件事上，两个版本的做法完全一样。

这一集我们学了

- ❶ 所有权：每块数据只有一个主人；`&` 是借来看看，`Vec<u8>` 是自己拥有
- ❷ 把规则写进类型：密钥是 `&[u8; 32]`，长度不对编译不过
- ❸ Result 和 `?`：会不会失败写在返回类型里，出错就提前 return
- ❹ trait 是接口，方法要先 use 进来；match 必须覆盖所有分支
- ❺ 测试也要被检验：期望值来自独立实现，再用变异测试确认它真能抓到错

下一集

下一集：把 Rust 送进浏览器，
WebAssembly、线性内存和 unsafe